Sealed Execution: Letting LLM Coding Agents Use Credentials Without Seeing Them

Mihir Gupta Stephen Keehn
Generalized Labs
https://github.com/Generalized-Labs/ironrun

Abstract

LLM coding agents run shell commands, and their output becomes model context. A credential printed by a debugging command can therefore reach the model provider and local transcripts. We present ironrun, an open-source runtime that injects secret values into child processes as environment variables or temporary files and filters known values and supported encodings before output reaches the agent. It combines value-blind Model Context Protocol interfaces, session-bound human authorization, a streaming redactor with whitespace-tolerant, fragment, and alignment-aware matching, and platform-specific OS controls with a hash-chained audit log. We state a chunking-invariance property for fixed patterns, bounded matching spans, successful writes, and a final flush, and present current regression case studies for approval integrity, agent-reachable surfaces, encoding structure, and non-vacuous containment tests. In a paired reproduction using the same checked harness on v0.5.0 and the reviewed runtime, 52 of 126 transformation/credential cases permit at least 50% case-insensitive contiguous recovery on the baseline, versus 28 on the reviewed runtime. For ordinary-tool transformations the counts are 21 of 72 versus 3 of 72; the three remaining cases involve a 12-byte password below fragment protection. A fresh current-only run reproduces these current counts. Shared-host Linux diagnostics additionally expose structured-value over-redaction and measure throughput and median spawn costs; they do not establish deployment leak rates, tail latency, or usability. Known-value redaction reduces accidental disclosure, but thresholds, partial disclosure, and unregistered transformations limit coverage. Authorization, protected executable code and inputs, and isolation must bound deliberate exfiltration.

1 Introduction

Coding agents such as Claude Code, Codex, and Cursor operate by issuing tool calls—most importantly, shell commands—and reading their output. That output is appended to the model's context window, transmitted to the model provider on every subsequent turn, and typically persisted by the agent host as a local transcript (Claude Code, for example, stores JSONL session logs under `~/.claude/projects/`). Anything a command prints is therefore *published*: to the provider, to the transcript store, and to anyone who later reads, shares, or screen-records the session.

Credentials reach output through utterly ordinary debugging. An agent asked to fix a failing integration test may run `printenv` to check configuration, `cat .env` to inspect it, `curl -v` to reproduce an HTTP error (which echoes the `Authorization` header), or simply rerun a test whose failure message embeds the database URL. Each of these discloses a secret the human never intended to share, and because transcripts cannot be retracted from a provider's logs, the only safe response is rotation. Practitioners report this happening repeatedly in normal use [14, 56], and a study of 17,022 agent skills reports debug-output exposure in 1,007 of 1,371 vulnerability issues (73.5%), where agent frameworks feed output into model context [16]. This percentage describes that study's issue category, not a population-wide rate of agent credential leakage. Once a value has entered context it also resurfaces in places transcript scrubbing cannot reach, such as reasoning traces [59].

Existing tools each cover part of the problem. Secret managers' `run` wrappers inject values into a child's environment [2, 24, 41]; 1Password's also conceals values in the child's output, but none of them mediates *which* commands an agent may run with which secrets. Per-tool file restrictions alone can leave indirect read paths [25, 53]; host-enforced filesystem controls can protect those paths as well [6, 7]. Credential brokers control authentication [1, 12]; filtering subsequent command output requires an output boundary. Proxy-substitution designs keep real values out of client processes. HTTP-specific proxies operate on traffic routed through their supported HTTP(S) paths [7, 26, 40]; protocol-aware brokers also support database and SSH connectors [20]. CI runners have long masked known secrets and their encodings in job logs [31, 32], and agent-side tools also inject secrets and scrub tool output by literal matching [27, 61]; Prismor additionally documents command policy and human approval [62, 63]. Our focus is the specific combination of transformation-aware streaming known-value redaction, session-bound authorization, and OS containment, evaluated at the agent's tool boundary for command-line programs.

Insight. The agent needs the *effect* of a credential—a passing test, a successful deploy—not its *value*. If the same component that resolves and injects secrets also mediates the command's output, then redaction becomes a problem over

known values rather than a detection problem over unknown ones. Known-value redaction is deterministic over a fixed pattern set and can be extended systematically to the encodings that ordinary tools apply. Coverage still depends on registration thresholds and overlapping-match behavior; it is not a universal confidentiality guarantee.

Contributions.

- A threat model for credential disclosure through agent tool output that separates *accidental* disclosure, *prompt-injected* agents acting through their tools, and hostile same-user code (Section 3).
- The design and implementation of ironrun, an MIT-licensed local runtime comprising value-blind MCP interfaces, session-bound tiered authorization with in-place continuation, a streaming redactor with a proved chunking-invariance property, and layered containment (Sections 4 and 5).
- Current regression case studies for approval integrity, agent-reachable interfaces, encoding structure, and non-vacuous containment tests, with executable checks linked to a pinned source snapshot (Section 6).
- An adversarial evaluation of known-value redaction at an agent's tool boundary: a 126-case leak corpus driven through the real execution path and attacked with a decoder battery, plus false-positive and overhead measurements, with an open harness (Section 7).

2 Background

Agent tool loops. An agent host repeatedly sends the conversation to a model, receives tool calls, executes them locally, and appends results. The Model Context Protocol (MCP) [52] standardizes how a host discovers and invokes local tool servers over stdio. Every byte a tool returns becomes model input, is retained for the rest of the session, and is commonly written to disk by the host.

Why output is the hard channel. Secrets can leave through three channels: *inputs* the agent writes (a value typed into a command line), *outputs* a command prints, and *side effects* (network, files). The first is solvable by never giving the agent the value; the third is the domain of sandboxes. The second is the subtle one: an agent that has never seen a credential can still cause it to be printed, merely by running the program that uses it. Rotation is the only remedy after the fact.

Scenarios. Three representative accidents motivate the design: (1) a connection failure prints a DSN with an embedded password; (2) `curl -v -u deploy:$TOKEN` prints `Authorization: Basic` followed by the base64 of `deploy:<token>`; (3) `docker compose config` prints the interpolated environment. In (2) the token never appears literally—it appears base64-encoded *at a byte offset that depends on the username*, a case that matters in Section 7.

3 Threat Model

Principals. The *operator* is the human developer; they are trusted, author the policy, and approve access. The *agent* is the model together with its host; it is trusted to act on the operator's behalf but may be mistaken or manipulated. *Child commands* are the operator's own build, test, and deploy programs, run on the agent's request. The *model provider* receives everything in context and is outside the operator's control.

Adversaries. We distinguish three:

A1 Accidental disclosure. An honest agent runs ordinary commands that happen to print secrets. This is the common case and the primary target.

A2 Manipulated agent. A prompt-injected or misaligned agent [33] tries to move a secret into its context or off the machine, but acts only through its tools: MCP calls, its native shell, and file edits.

A3 Hostile same-user code. Arbitrary native code running as the operator's UID. It can read the user's keychain, attach to processes where the OS permits, and rewrite user-owned files. A3 is out of scope; containing it requires a separate OS user, container, or VM.

Goals.

G1 Value-blind interfaces. Agent tools expose no plaintext secret-value or vault-key fields. Trusted local input and explicit CI mask emission use separate operator channels (Section 4.1).

G2 Output filtering. The output boundary replaces recognized literal and derived matches under the defined matching policy; thresholds, overlaps, and transformations limit confidentiality.

G3 Least authority. Saved commands restrict argv and secret selection. Optional leases and trusted-workspace grants bind authority to a session, environment, and expiry; revocation is checked on the next call.

G4 Containment. Requested network isolation and process-group time limits apply on supported platforms. Linux additionally provides syscall filtering and a core-dump limit; macOS does not implement those Linux-specific controls.

G5 Accountability. With auditing enabled, successful appends retain command metadata and secret *names* in a hash chain. Availability and integrity limits are explicit in Section 4.6.

Non-goals. ironrun does not attempt to stop an authorized process that *deliberately* transforms a secret with an arbitrary function before printing it (Observation 1); does not defend against A3; does not remove data from provider logs; and does not guarantee physical erasure of temporary file secrets on SSDs or copy-on-write filesystems.

A note on A2 and the native shell. Agent hosts typically give the agent its own unrestricted shell in addition to

any MCP server. ironrun removes secrets from the agent's ambient environment and from the repository, which defeats A1 and casual A2 (`printenv`, `cat .env`). But a determined A2 with an unrestricted same-user shell approaches A3: on macOS, for instance, a keychain item created through `/usr/bin/security` is readable by any process that invokes the same binary. Deployments that face A2 must therefore pair ironrun with the host's own command restrictions or sandbox, or run ironrun's state under a separate user. We return to this in Section 8.

4 Design

Figure 1 shows the architecture. CLI, MCP, TUI, and local Unix-socket API paths share an execution core for secret resolution, containment, redaction, and audit. Authorization remains each caller's responsibility; sharing the execution core does not by itself establish equivalent authorization checks.

4.1 Value-blind interfaces

The agent-facing MCP server exposes twelve tools (Table 1). Their schemas solicit aliases and metadata, not plaintext secret values. String fields can still contain arbitrary text; the schema is not a content confidentiality proof. `request_secret`, for example, accepts a declared alias and a reason; the human fulfils it through masked local input. If a workflow requires pasting through chat, the operator instead produces an *encrypted capsule* (`ir1....`): AES-256-GCM ciphertext under a per-project capsule key, bound by its authenticated data to the project and by its payload to the requesting MCP session, request, alias, and a ≤10-minute expiry, and usable once. The plaintext never enters the transcript.

The explicit operator flag `–emit-github-masks` is a separate CI channel: it prints mask-registration commands containing escaped values for the CI parser. It must not be routed to an agent transcript or treated as a value-blind CLI response.

4.2 Authorization mechanisms

ironrun combines four authorization mechanisms:

1. **Strict commands.** The policy file lists exact argv vectors and the secrets each receives. The agent calls `run_sealed` with a command ID; no shell expansion is performed by the wrapper, and argv[0] may not be a shell. The policy is re-read on every call.
2. **Leases.** With `require_agent_leases`, a strict command also needs an unexpired, unrevoked lease bound to the MCP session, environment, and command set (default 1 h, maximum 24 h).
3. **Trusted workspace sessions.** For everyday development, the operator grants one MCP session arbitrary-argv execution against one project environment for a bounded time (default 2 h). `staging` and `prod` require separate grants.

Tool	Returns (no secret-value fields)
`list_commands`	IDs and argv of approved commands
`run_sealed`	exit code, duration, redacted stdout/stderr
`request_workspace_access`	request ID for a trusted session
`workspace_status`	grant state, environment names
`request_lease` / `lease_status`	lease request / state
`revoke_own_lease`	revocation of the caller's own lease
`propose_command`	staged proposal ID (runs nothing)
`request_secret`	request ID (no value field)
`claim_capsule`	success flag; stores ciphertext's value locally
`list_environments`	names and key names only
`validate_policy`	provider and command count

Table 1: The agent-facing MCP surface. A registry guard rejects removed key-export tools. A native-store-dependent fixture probe checks sampled handler results for vault key material; it may skip on headless hosts.

4. **Proposals.** An agent may stage a new strict command with `propose_command`; it is inert until the operator approves it, at which point it is merged into the policy.

Session identifiers are 64-bit values from `crypto/rand`, minted per MCP server process and never accepted from the agent, so a grant cannot outlive the server instance it was issued to.

Continuation. A blocked call does not fail; it creates (or reuses) a deduplicated, value-blind request and waits on persisted state, re-validating authorization every 100 ms until the request's deadline (Fig. 2). When the operator approves in the TUI, the original call resumes without a separate approve-and-report exchange in chat.

4.3 Resolution and storage

Secrets come either from an existing manager through its CLI (1Password, HashiCorp Vault, Doppler, Infisical), from an owner-only env file outside the repository, or from ironrun's own encrypted vault (Fig. 3). The vault keeps a random 32-byte root key in the OS credential manager. Each environment (a *scope*) is sealed under a fresh 32-byte data key on every write; the data key is wrapped by the root key with AES-256-GCM, and both encryptions bind version, purpose, project, and scope in their authenticated data, so ciphertexts cannot be moved between environments or projects. The whole manifest carries an HMAC-SHA256 under the root key and is committed by write-temp, fsync, rename, and directory fsync. Only names, kinds, and timestamps are stored unencrypted, under `.ironrun/` in the project.

File secrets (e.g. a service-account JSON) are materialized only for the duration of a run, in a fresh 0700 directory outside the repository, with `O_EXCL` 0600 files and validated basenames; only the path is injected, and the contents are registered for redaction.

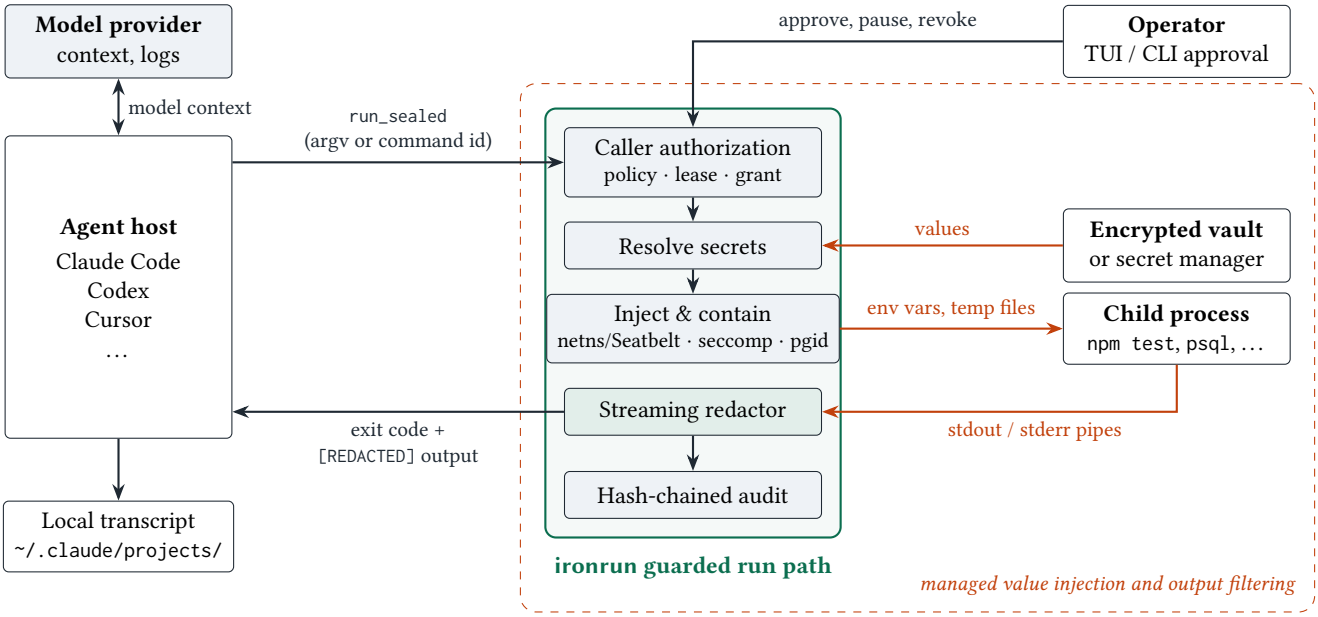


Figure 1: The MCP run path. Orange arrows show value delivery to the child and its output pipes. Caller authorization precedes shared execution. The tool returns an exit code and output filtered under the stated matching policy; MCP approval requires an operator action.

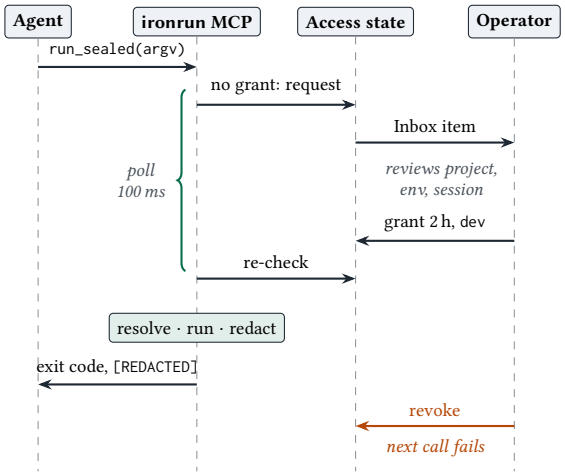


Figure 2: Request, approval, and in-place continuation. The waiting call is never restarted, and approval happens only on the operator's side.

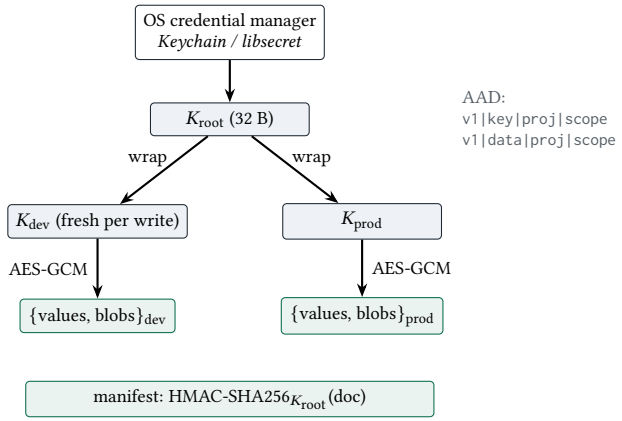


Figure 3: Vault key hierarchy. Rewriting a scope rotates its data key; AAD binds every ciphertext to its project and scope.

4.4 Streaming known-value redaction

Each of the child's output streams passes through its own redacting writer. Let V be the set of injected values (and file-secret contents) of at least 4 bytes. The writer registers a pattern set $P = V \cup \bigcup_{v \in V, |v| \geq 8} \mathrm{Enc}(v)$, where Enc derives the encodings that ordinary tools apply: base64 (standard and URL alphabets, padded and raw), lower- and upper-case hex, percent-encoding in the styles of Go, JavaScript, and Python (with upper- and lower-case escapes), JSON string escaping, second-level nestings (e.g. base64 of base64), base64 wrapped at 64 and 76 columns, and—crucially for the Basic-auth scenario—the *alignment cores* of base64 at byte offsets 1 and 2 (Section 4.4.1).

Matching tiers. At each buffer position the writer tries, in order:

1. **Exact**, longest pattern first, using a first-byte index.
2. **Gap-tolerant**: the pattern's bytes appear in order with interleaved whitespace or control bytes ($\leq$ `0x20`, `0x7F`), which covers terminal line wrapping, `\r \n` injection, UTF-16LE null interleaving, and one-character-per-line output. The consumed span is capped at the global span m defined below.
3. **Fragment**: for patterns with 24 to 65,536 non-gap bytes, every 12-byte window of the pattern's gap-free projection is indexed. Twelve output bytes that match a window, in order and with any gap bytes between them, are extended forward over the rest of the pattern and redacted. This catches truncated and partial prints (one line of a PEM key) and line-oriented dumps (`xxd`, `od -c`) whose offset columns cut a value into per-line pieces.

Algorithm 1 Streaming redaction (one output stream)

```
1:  procedure Write(x)
2:      B ← B ‖ x; Scan(m − 1)
3:  procedure Flush
4:      Scan(0)
5:  procedure Scan(h)
6:      while |B| > h do
7:          i ← 0
8:          while i < |B| − h and Match(B, i) = 0 do
9:              i ← i + 1
10:         ℓ ← 0
11:         if i < |B| − h then
12:             ℓ ← Match(B, i)
13:         Emit(B[0:i])
14:         if ℓ = 0 then
15:             B ← B[i:]; break
16:         Emit([REDACTED]); B ← B[i + ℓ:]
17: function Match(B, i)                    ▷ bytes consumed, ≤ m
18:     return first nonzero of Exact, GapTolerant, Fragment
    at B[i:]
```

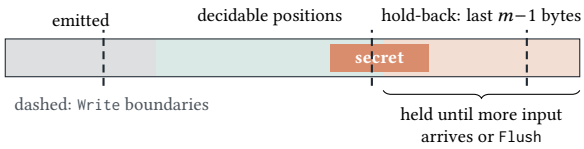


Figure 4: The scanner only decides positions that have at least $m-1$ bytes of lookahead, so a secret that straddles one or more `Write` boundaries is still seen whole.

A match is replaced by `[REDACTED]`; unmatched bytes are emitted only once no pattern could still begin at them.

Bounded hold-back. Let $m = \max(\{1\} \cup \{2|p| + 32 : p \in P\})$ bound the span any tier may consume, including when P is empty. The writer withholds the last $m - 1$ bytes of its buffer until more input arrives or the stream is flushed (Algorithm 1 and Fig. 4).

Property 1 (Chunking invariance)**.** *For fixed patterns and matching indexes, complete successful underlying writes, and a single final* `Flush`*, any partition of input x into* `Write` *calls emits the same output as* `Write`(x) *followed by* `Flush`*.*

Proof sketch. Scan decides position i only if $|B| - i > m - 1$, i.e. at least m bytes starting at i are buffered. Every tier's decision depends on at most m bytes from i: exact matching reads $|p| < m$ bytes; gap-tolerant matching accepts spans of at most m; fragment lookup and extension are explicitly limited to m bytes. Hence Match(B, i) depends only on bytes already present and is identical to its value on the complete input. Positions are decided left to right in the same order in both executions, and `Flush` decides the remaining positions with the full suffix present. Induction on decided positions gives equal output. □

The cost of this guarantee is latency: up to $m - 1$ bytes (a few hundred bytes for typical tokens) are delayed until the next write or process exit. This bounds buffered bytes, not wall-clock delay: a quiet interactive process may leave output buffered until it prints more data or exits. Output order is preserved.

4.4.1 Base64 alignment

Base64 maps each 3-byte group to 4 characters. When a secret s of n bytes is encoded as part of a larger message, the characters that depend *only* on s are determined by s's offset modulo 3. With $k \in \{0, 1, 2\}$ bytes of preceding context, they are

$$\mathrm{core}_k(s) = \mathrm{b64}(0^k \| s)\left[\lceil 8k/6 \rceil \,:\, \lfloor 8(k+n)/6 \rfloor\right],$$

where the slice retains complete sextets whose bits come only from s; the trailing bound excludes a final sextet if it mixes in following context. Ordinary $\mathrm{b64}(s)$ is not a context-independent search pattern: preceding bytes change bit alignment, and following bytes can change padding or a boundary sextet. HTTP Basic authentication (`user:token`) and `printenv | base64` can introduce such context. Registering the three sufficiently long cores covers these alignments, subject to the matching thresholds and overlap policy. The technique is not new—the GitHub Actions runner has registered shifted base64 forms since 2019 [31]—and the current encoding case study in Section 6 checks these alignments.

4.5 Containment

The following controls reduce selected side effects. Requested network isolation and the Linux child shim refuse execution when their setup fails; their scope does not include filesystem or general interprocess isolation.

- **Network.** Secret-bearing commands get no network unless the policy opts out. On Linux ironrun enters fresh user and network namespaces [46, 47] (`CLONE_NEWUSER|CLONE_NEWNET` with an identity UID/GID map, so no privilege is needed); on macOS it wraps the child in a Seatbelt [13] profile that denies only `network*`, invoking `/usr/bin/sandbox-exec` [8] by absolute path. Unsupported platforms refuse to run.
- **Syscalls.** On Linux the child is started through a re-exec shim that sets `NO_NEW_PRIVS`, installs a seccomp-BPF filter [70] that checks the architecture, rejects the x32 ABI, and returns `EPERM` for `ptrace`, `process_vm_readv`, `process_vm_writev`, `kcmp`, `perf_event_open`, `bpf`, and `userfaultfd`, then sets `RLIMIT_CORE=0` and execs the target in place.
- **Lifetime.** The child runs in its own process group; on timeout, cancellation, or a signal to ironrun, the whole group is killed, and any group member left after the child exits is killed too. Descendants that detach with `setsid` can escape this group and retain injected values; this is not whole-process-tree containment. Temporary file secrets are removed during normal error, cancellation, timeout, and success cleanup, with stale-directory recovery after crashes.

- **Environment.** The inherited environment is stripped of loader, shell-startup, interpreter, and tool hijack variables (`LD_*`, `DYLD_*`, `BASH_ENV`, `NODE_OPTIONS`, `PYTHON*`, `GIT_CONFIG_*`, `GOFLAGS`, ...) before secrets are added.
- **CI.** In GitHub Actions, GitLab CI, CircleCI, and Jenkins, runs triggered by fork pull requests (and `pull_request_target`, the "pwn request" pattern [44, 49], without an explicit operator flag) are refused before any secret is resolved.
- **Operator-only switches.** Explicit overrides for seccomp, the core-dump seal, entropy scan, and `pull_request_target` use command-line flags rather than environment-variable switches. An unrestricted agent shell can still invoke such flags; host restrictions must protect their use.

4.6 Audit

When auditing is enabled and the child starts, the runner attempts to append a JSONL record: command, argv scrubbed for registered literal and encoded values, secret names and kinds, redaction count, exit status, and sandbox outcome. Each record hashes the previous record's hash. Opening the logger verifies the existing chain and refuses detected corruption; a long-lived server does not replay it before every run. Later append or sync errors warn without failing the command, and policy can disable auditing. `ironrun audit verify` detects inconsistent edits or ordering when hashes have not been recomputed. This unkeyed chain does not provide the authenticated or forward-secure history of secure logging constructions [68]. Without a protected external anchor [19], an attacker can rewrite and rehash the whole file or truncate its tail (Section 8).

5 Implementation

ironrun is implemented in Go, with unit tests, differential tests, and redactor fuzzing. Release binaries are built with `CGO_ENABLED=0`. The MCP server uses `mcp-go`. Platforms are macOS and Linux on amd64 and arm64; Windows is unsupported because its secret-entry paths would require plaintext console input. Builds use `-trimpath`, commit-date timestamps, and pinned modules. Releases carry SHA-256 checksums, an SBOM, and keyless Sigstore bundles [57]. The installer and GitHub Action verify checksums and bundles; npm verifies archive and binary hashes against its bundled manifest, without independent Sigstore verification. CI runs OpenSSF Scorecard, `govulncheck`, and scheduled fuzzing.

6 Regression Case Studies

We examine executable regression checks at source commit `a4ab72e475bb6d3ed548e0dd7569294a1b3a0e8d`. These cases specify properties of that snapshot, rather than historical defect counts or severity assessments. `paper/REPRODUCING.md` identifies the tests and commands.

Lesson 1: human review depends on the enforced artifact. The approval regression tests insert proposal-controlled newlines, Unicode line separators, carriage returns, and YAML-shaped text into reasons, references, names, and argv. They parse the resulting policy and reject any added command beyond the intended proposal. A separate check changes proposal contents after computing the review digest and requires approval to fail without changing the policy or consuming the replacement. These checks exercise serialization and content binding; they do not measure whether developers understand the approval interface. The general rule is that human approval must be bound to the parsed artifact that will be enforced, with unambiguous argv boundaries.

Lesson 2: value-blindness must cover every reachable surface. A CLI regression asserts that `ironrun env share` does not resolve to a command. The CI mask regression uses a synthetic multi-line value and requires each output line to be a mask command, with each secret line separately registered. These checks cover CLI reachability and a line-oriented protocol where printing a multi-line value as one command would expose later lines. An MCP-only interface review would not cover these surfaces.

Lesson 3: encodings have structure. The redaction shape tests exercise Basic-auth base64 with varying prefixes and byte alignments, JSON/percent escaping, hexdump columns, and partial file-secret output. They complement encoding enumeration with the formatting conventions of specific tools. The paired leakage experiment in Section 7 supplies baseline/current evidence for that corpus; these current regression checks do not independently reconstruct the history of each change.

Lesson 4: a denied operation must actually be attempted. The network regression requires a distinct `DIAL_BLOCKED` marker as well as nonzero exit, so a process that fails before dialing does not count as successful isolation. A macOS-specific check separately requires a sandboxed fork/exec to return `child-ok`. Checks may skip when the platform or isolation capability is unavailable; a skip is not positive containment evidence. The Linux diagnostics below do not validate the macOS-only path.

7 Evaluation

We ask: **RQ1** how much of a secret reaches the agent under realistic and adversarial output transformations; **RQ2** how often benign output is redacted; **RQ3** what the runtime costs; and **RQ4** whether streaming behaves exactly like batch processing. We separate the paired leakage reproduction recorded on October 21, 2026 from fresh current-only diagnostics on October 22. The latter use source commit `a4ab72e475bb6d3ed548e0dd7569294a1b3a0e8d`, Go 1.26.10, and the existing `paper/eval` harness on a shared Linux amd64

Transformation	Class	v0.5.0	Reviewed
`printenv` line	acc.	0/6	0/6
`set -x` trace	acc.	0/6	0/6
JSON (Go encoder)	acc.	1/6	**0/6**
URL query parameter	acc.	0/6	0/6
HTTP Basic auth	acc.	6/6	**0/6**
k8s Secret (base64)	acc.	0/6	0/6
`printenv | base64`	acc.	6/6	**0/6**
`xxd` hexdump	acc.	6/6	**1/6**
ANSI-highlighted	acc.	1/6	1/6
pty wrap (CRLF)	acc.	0/6	0/6
UTF-16LE	acc.	0/6	0/6
truncated (first 60%)	acc.	1/6	1/6
hex	del.	0/6	0/6
base64 at offset 1	del.	6/6	**0/6**
double base64	del.	0/6	0/6
gzip | base64	del.	6/6	6/6
one char per line	del.	0/6	0/6
split with separator	del.	1/6	1/6
reversed	del.	6/6	6/6
ROT13	del.	6/6	6/6
upper-cased	del.	6/6	6/6
Accidental		**21/72**	**3/72**
All		**52/126**	**28/126**

Table 2: Secrets (of 6) of which ≥50% is recoverable from agent-visible output in the October 21 paired reproduction using the same checked harness. *acc.*: ordinary-tool transformations; *del.*: deliberate. A separate October 22 current-only run reproduces 28/126 and 3/72.

host (Intel Xeon Platinum 8573C, three visible CPUs, a two-CPU cgroup quota, 8 GiB memory limit). These synthetic microbenchmarks do not estimate production incidence or developer usability. Artifact identities and reproduction commands are in `paper/REPRODUCING.md`.

Method (RQ1). We generate six synthetic credentials shaped like real formats: a Stripe-style key (32 B), a GitHub PAT (40 B), an AWS-style secret (40 B), a Postgres URL with an embedded password (75 B), a 12-byte password, and a JWT (113 B). For each of 21 transformations we write the transformed output to a file and have a real child (`cat`) print it through `runner.Run` with the secret injected—the same injection, pipe, registration, and redaction path a sealed run uses. We then attack the agent-visible output with a decoder battery: identity, gap stripping, case folding, reversal, ROT13, base64 and hex decoding at every alignment (with nesting and gunzip), URL and JSON unescaping, and hexdump column reassembly. *Recovered* is the longest contiguous run matched after case folding by any decoder; a case *leaks* if at least 50% of the secret matches. This measures case-insensitive contiguous disclosure, not necessarily a usable credential. A shorter recovered component can still expose the entire password inside a structured URL. Rates describe this deterministic corpus, not a real-world probability of leakage.

Results (RQ1). Table 2 shows that v0.5.0 already handled literal prints, xtrace, URL parameters, aligned base64, wrapping, CRLF, UTF-16, and nested encodings, but leaked every secret under three *accidental* transformations—Basic auth, `printenv | base64`, and `xxd`—for the alignment and formatting reasons of Section 6. In the reviewed runtime, no accidental case in this corpus with a credential of at least 24 bytes meets the 50% threshold. This is not a claim of zero partial disclosure. The three remaining accidental cases all involve the 12-byte password, which is below the fragment threshold: a 60% prefix, a color escape in the middle of the value, and the `xxd` ASCII column, which prints it in two pieces.

Four deliberate transformations recover every credential in this corpus. More generally:

Observation 1. *Known-value matching alone does not guarantee confidentiality against arbitrary transformations. An authorized process can disclose a secret through an invertible representation if the emitted representation avoids every registered match. Being unregistered is not sufficient: a transformation can still produce a registered pattern (for example, reversal of a palindrome).*

Redaction is therefore a defense against A1, not against an A2 that can choose the program. Against A2, ironrun relies on strict commands with host-enforced protection of executable code and inputs. Network isolation reduces network egress but does not isolate writable files or local IPC; those channels require additional host controls. The warn-only entropy scan flagged 21 of the 28 threshold leaks, with warnings on 6 of the 98 cases below that threshold. The latter need not be false alarms: partial or component recovery can remain security-relevant below 50%. The gzip/base64 case is instructive: the emitted representation looks unlike the credential but is readily decoded, which is why it is included.

RQ2: false positives. The fresh current-only run processes a frozen 1,209,966-byte corpus: 209 Go and Markdown files in the source snapshot, plus 1,000 generated log lines sharing structure with a connection URL. The ordered input manifest, per-file hashes, concatenated corpus hash, and raw outputs are retained. Each of the six synthetic credentials is registered separately with its full encoding set (Table 3). Five formats cause no replacements; the Postgres URL causes 1,002. This is a replacement-event count, not a byte- or line-level precision measure. The generated log lines deliberately share non-secret URL windows (e.g. `db.internal:5432/app`) that trigger fragment matching; they dominate the structured-value result. No fresh baseline false-positive run was performed, so this table does not compare versions. The result motivates component-aware registration but does not establish an optimal mitigation.

RQ3: shared-host runtime diagnostics. For 200 successful samples per mode, the fresh run reports median wall-clock spawn times of 1.043 ms for direct `true`, 2.561 ms through `runner.Run` with one synthetic secret, and 3.342 ms through the runner with Linux network isolation. The harness runs

Credential	Patterns	FP events
Stripe key	18	0
GitHub PAT	18	0
AWS secret	18	0
Postgres URL	22	1,002
password (12 B)	14	0
JWT	22	0

Table 3: Fresh current-only replacement counts on the frozen 1,209,966-byte benign corpus. The generated DSN logs dominate the Postgres result; these counts do not measure deployment precision.

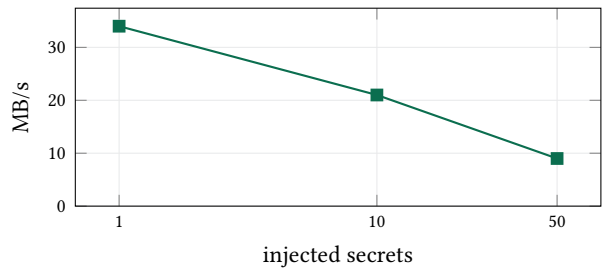


Figure 5: Current-only shared-host scanning diagnostic: best of five repetitions, excluding pattern generation and writer construction.

these modes in fixed order within each repetition; differences between their medians are not a paired per-command overhead estimate. These runner microbenchmarks exclude provider lookups, audit writes, session authorization, and the production seccomp filter. The run records medians only, so it cannot establish p95/p99 latency or time to first visible output. A preceding attempt inside an additional filesystem sandbox failed at the first isolated spawn with a read-only-filesystem error; its failure is retained separately. The complete run used the same fixed binary outside that additional sandbox, with the runtime's isolation still enabled.

Figure 5 reports 34, 21, and 9 MB/s for 1, 10, and 50 synthetic values (18, 180, and 900 registered patterns). Each point is the fastest of five repetitions scanning approximately 4 MiB of repeated neutral log text in 32 KiB writes, including the final flush but excluding encoding generation and writer construction. The generated values share padded prefixes, and the payload contains no matches. This diagnostic does not characterize long values, match-heavy output, small writes, memory costs, or interactive delivery delay.

RQ4: streaming equivalence. The current source contains `TestStreamingMatchesBatch`, a differential check over 3,000 deterministically generated inputs comprising registered patterns, fragments, gaps, and noise. It compares a single write with chunks of 1–13 bytes followed by a final flush. `FuzzRedact` additionally checks streaming/batch agreement and fragment coverage for its fixed pattern set. These are executable regression specifications for Property 1; they do not reconstruct the original historical run or constitute exhaustive testing. The fresh `paper/eval` runs do not execute these tests. The reproduction guide lists their commands and platform-dependent checks.

8 Discussion and Limitations

Substitution avoids value exposure. A child that receives only a placeholder cannot print a credential it never held. HTTP-specific proxies substitute values on supported outbound HTTP(S) paths [7, 40, 58]; protocol-aware brokers also support database and SSH connections [20]. Where a broker supports the workflow, keeping values out of the child is preferable. Local signing, direct file consumers, and unsupported connectors can still require credentials in the process; output filtering addresses that remaining boundary and can compose with brokers.

Redaction is a safety net, not a sandbox. Observation 1 is fundamental. ironrun's strongest configuration—strict commands, leases, and no network—restricts argv. Restricting what the program computes additionally requires protecting its code, dependencies, and inputs from agent edits. Trusted workspace sessions trade this away for convenience: the agent chooses argv, receives every entry of the environment in the child, and could deliberately encode a secret. We document trusted sessions as protecting context and logs from accidents, not as containment.

State integrity against A2. Authorization state (`.ironrun/`, `ironrun.yml`) lives in the project, where an agent with file-edit tools can write it, and a trusted session can run ironrun itself to change access. Same-user storage cannot protect itself from same-user code. The principled remedy is to move approvals and state behind a broker running as a different user, with the agent host's own file and command restrictions guarding the checkout in the meantime.

Credential-store reachability. Keychain items created through `/usr/bin/security` are readable by any same-user process that invokes it, and the legacy file store keeps its key beside its ciphertext. Both are consistent with the A3 exclusion but matter for A2 with an unrestricted shell.

Audit rewriting and vault rollback. The unkeyed audit chain can be rewritten and rehashed by an attacker who controls the whole file. The vault manifest MAC authenticates modifications under its key but does not prevent replay of an older valid state. Detecting replacement, truncation, or rollback requires an independently protected anchor and monotonic revision state; same-user credential storage alone does not establish that trust boundary.

Partial and structured secrets. A short secret (<24 B) printed partially—say its first seven characters—is not fragment-indexed, and a structured secret's boilerplate windows cause over-redaction (Table 3). Both trace to treating every 12-byte window alike; weighting windows by estimated entropy, or decomposing URLs into their userinfo component, are natural refinements. Separately, when two distinct registered values overlap in the output, tier priority

and earlier consumption can prevent recognition of the later value. The surviving suffix depends on value length and registered forms; there is no general eleven-byte bound. Synthetic overlapping values reproduce this behavior, including cases where most of a later value survives.

Platform. macOS's `sandbox-exec` is deprecated though functional; Windows is unsupported. Provider-side retention of anything that *did* reach context is outside ironrun's reach.

9 Related Work

Prompt injection and agent security. Indirect prompt injection [33, 60] and its formalization and benchmarks [21, 48, 80] establish that an agent's behavior can be steered by data it reads, and adaptive attacks defeat most model-level defenses [55]. Willison's "lethal trifecta" [74] and Meta's "rule of two" [51] frame exfiltration as the conjunction of private data, untrusted input, and an outbound channel. ironrun reduces direct value exposure at the tool boundary and can restrict network egress. It does not remove filesystem or local IPC channels from the child.

System-level defenses. CaMeL [22], design patterns for injection-resistant agents [11], information-flow control [18, 75], privilege policies [69], and execution isolation [76], building on the dual-LLM pattern [73], constrain what a compromised planner can do. ironrun is narrower and deterministic: one data class (credentials), one sink (tool output), no change to the model or planner. Its redactor is a fixed declassification rule at a single boundary, and its strict mode corresponds to the action-selector pattern at the shell level. A recent systematization of execution security for coding agents finds deny-lists failing at high rates [66], consistent with our allowlist-first design.

MCP security. Measurement and attack studies of the MCP ecosystem [15, 34–36, 81, 82] document tool poisoning [42, 72], credential theft through MCP-connected tools [65], bidirectional data-flow risks [37], and context leaking into tool arguments of coding agents [78]. ironrun sits on the return path of a local stdio server and assumes a possibly poisoned agent.

Secret leakage and detection. Secrets leak widely in public code [50], code models can memorize them [38], and vendor data suggest AI-assisted commits leak more often than average [28, 30]. Pattern-based detectors trade precision for recall [9, 10]; ironrun sidesteps detection by matching only values it injected. Closest in the literature, Kenney et al. evaluate a vault-mediated boundary for API connectors and conclude that output redaction is a separate, necessary layer [43], and SlotGuard substitutes *detected* secrets at the transcript boundary [77]. Chen et al. identify stdout-to-context as the dominant leak path in agent skills without proposing a runtime fix [16].

CI log masking. The GitHub Actions runner masks registered secrets and automatically derived forms—base64 at all three alignments, URI, JSON, XML, and shell escapes—in job logs [31], while noting that masking relies on exact matches and is not guaranteed [32]. ironrun adopts this idea for an interactive, streaming setting and adds whitespace-tolerant and fragment matching. Its GitHub mask regression checks that multi-line values are escaped before emission to the line-oriented mask protocol (Section 6).

Industry tools. Run wrappers inject secrets [2, 24, 41], and `op run` also conceals them in output. Agent access products broker credentials [1, 12]; Doppler's MCP server deliberately exposes values to agents [23] and Infisical's can mask values in its own responses [39]. Proxy substitution keeps values out of processes for HTTP(S) [7, 26, 40, 58]; Secretless Broker also supports database and SSH connectors [20]. The nearest tools include fnox's MCP server, with tool/secret allowlists and `exec` that accepts agent-chosen argv, injects secrets, and masks literal values [27]. Prismor's Cloak substitutes placeholders before execution and filters literal output; its current documentation also describes command policy and human approval [61–63]. These documented controls overlap with ironrun; we do not establish their historical availability or equivalent enforcement semantics. Pattern-based scanners and gateways inspect tool traffic [29, 64, 67, 71]. LangWatch filters traces before storage or upload [45], and other tools scrub existing agent histories [54]. Agent hosts now expose hooks that can rewrite tool output [5] and ship their own sandboxes [4, 7]. ironrun focuses on combining encoding-, alignment-, whitespace-, and fragment-aware known-value redaction with session-bound authorization and OS containment for arbitrary programs, and in evaluating that redaction adversarially.

Isolation and integrity primitives. Heavier isolation via user-space kernels or microVMs [3, 79] and unprivileged sandboxes [17] would strengthen containment at a cost. ironrun ships as one Go binary, using Linux namespaces/seccomp or macOS Seatbelt; native secure-storage and platform sandbox tools remain host dependencies. Release signing follows Sigstore [57].

10 Conclusion

Coding agents need to use credentials far more often than they need to know them. ironrun filters known secret values and supported encodings from intercepted child-process output. Its protection is limited by matching thresholds, transformations, and the host-enforced execution boundary. A streaming known-value redactor with a provable chunking property and session-bound human authorization with in-place continuation describe separate parts of the boundary. The paired experiment measures redaction coverage; platform controls provide additional, scoped restrictions. Regression case studies expose approval serial-

ization, reachable interfaces, encoding structure, and non-vacuous test assertions as concrete review targets. Paired leakage reproduction and fresh shared-host diagnostics make some limits measurable; component-level precision/recall, tail and delivery latency, and developer usability remain unmeasured. ironrun is available under the MIT license at `https://github.com/Generalized-Labs/ironrun`.

Availability. The evaluation harness (`go run ./paper/eval`) runs leakage, false-positive, and performance experiments; `make -C paper` builds this document. `paper/REPRODUCING.md` gives commands, source/artifact hashes, and the checked harness for the pinned v0.5.0 baseline. Paired leakage results and host records are retained under `docs/validation/2026-10-21/`. The separate October 22 current-only diagnostics, frozen corpus, and both performance attempts are retained under `docs/validation/2026-10-22/fresh-pilot/`.

Disclosure. Code review, experiment execution, and manuscript preparation were assisted by LLM-based tools under the authors' direction. The evidence reported here is limited to the retained reproductions, current regression specifications, and source review described above.

References

[1] 1Password. 1Password launches unified access platform to help companies securely deploy AI agents. Press release, March 2026. URL `https://1password.com/press/2026/mar/1password-unified-access`. Published 2026-03-17. Accessed 2026-10-09.

[2] 1Password. `op run` — 1Password CLI reference, 2026. URL `https://www.1password.dev/cli/reference/commands/run/`. Accessed 2026-10-09.

[3] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020. URL `https://www.usenix.org/conference/nsdi20/presentation/agache`.

[4] Anthropic. sandbox-runtime: lightweight OS-level filesystem and network sandboxing for arbitrary processes. GitHub repository anthropics/sandbox-runtime, 2025. URL `https://github.com/anthropics/sandbox-runtime`. Accessed 2026-10-09.

[5] Anthropic. Hooks reference (PostToolUse `updatedToolOutput`). Claude Code documentation, 2026. URL `https://code.claude.com/docs/en/hooks`. Accessed 2026-10-09.

[6] Anthropic. Configure permissions. Claude Code documentation, 2026. URL `https://code.claude.com/docs/en/permissions`. Accessed 2026-10-09.

[7] Anthropic. Configure the sandboxed Bash tool. Claude Code documentation, 2026. URL `https://code.claude.com/docs/en/sandboxing`. Sections "Protect credentials" and "Mask credentials". Accessed 2026-10-09.

[8] Apple Inc. sandbox-exec(1) — execute within a sandbox (DEPRECATED). macOS General Commands Manual. URL `https://keith.github.io/xcode-man-pages/sandbox-exec.1.html`. Manual mirror accessed 2026-10-24.

[9] Setu Kumar Basak, Jamison Cox, Bradley Reaves, and Laurie Williams. A comparative study of software secrets reporting by secret detection tools. In *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12, 2023. doi: 10.1109/ESEM56168.2023.10304853.

[10] Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. SecretBench: A dataset of software secrets. In *IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 347–351, 2023. doi: 10.1109/MSR59073.2023.00053.

[11] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections, 2025. URL `https://arxiv.org/abs/2506.08837`.

[12] Bitwarden. Shadow AI agents are already in your organization. Here's how to secure their credential access with Agent Access SDK. Bitwarden blog, June 2026. URL `https://bitwarden.com/blog/shadow-ai-agents-how-to-secure-credential-access-with-agent-access-sdk/`. Code: https://github.com/bitwarden/agent-access. Accessed 2026-10-09.

[13] Dionysus Blazakis. The apple sandbox. Black Hat DC 2011 (whitepaper), 2011. URL `https://media.blackhat.com/bh-dc-11/Blazakis/BlackHat_DC_2011_Blazakis_Apple_Sandbox-wp.pdf`. Accessed 2026-10-09.

[14] castours-dev. SECURITY: Claude Code reveals secret keys and credentials in terminal output despite explicit prohibitions. GitHub issue anthropics/claude-code#32523, 2026. URL `https://github.com/anthropics/claude-code/issues/32523`. Opened 2026-03-09; closed. Accessed 2026-10-09.

[15] Pei Chen, Baichao An, Mengying Wu, Binwang Wan, Geng Hong, Jinsong Chen, Xudong Pan, Jiarun Dai, and Min Yang. Rethinking MCP security: A large-scale study of runtime MCP servers and security scanner reliability, 2026. URL `https://arxiv.org/abs/2607.11086`.

[16] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. How your credentials are leaked by LLM agent skills: An empirical study, 2026. URL https://arxiv.org/html/2604.03070v2. Version 2, 19 June 2026. Accessed 2026-10-20.

[17] containers/bubblewrap contributors. bubblewrap: low-level unprivileged sandboxing tool. GitHub repository, 2016. URL https://github.com/containers/bubblewrap. Accessed 2026-10-09.

[18] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control, 2025. URL https://arxiv.org/abs/2505.23643.

[19] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*, 2009. URL https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident.

[20] CyberArk. Secretless broker. GitHub repository cyberark/secretless-broker, 2017. URL https://github.com/cyberark/secretless-broker. Accessed 2026-10-09.

[21] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[22] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL https://arxiv.org/abs/2503.18813.

[23] Doppler. Doppler MCP server. GitHub repository DopplerHQ/mcp-server, 2025. URL https://github.com/DopplerHQ/mcp-server. Experimental. Accessed 2026-10-09.

[24] Doppler. CLI guide (`doppler run`), 2026. URL https://docs.doppler.com/docs/cli. Accessed 2026-10-09.

[25] Martin Paul Eve. Claude Code can consume, transmit, and compromise your .env files even if you tell it not to. Blog post, April 2026. URL https://eve.gd/2026/04/19/claude-code-can-consume-transmit-and-compromise-your-env-files-even-if-you-tell-it-not-to/. Accessed 2026-10-09.

[26] Fly.io. tokenizer: HTTP proxy that injects 3rd party credentials into requests. GitHub repository superfly/tokenizer, 2023. URL https://github.com/superfly/tokenizer. Accessed 2026-10-09.

[27] fnox project. fnox: MCP server guide. Documentation for jdx/fnox, 2026. URL https://fnox.jdx.dev/guide/mcp.html. Source checked at commit 9a51b390fca0a060a7eee3003ff3e99ea4200272. Accessed 2026-10-21.

[28] GitGuardian. The state of secrets sprawl 2025, 2025. URL https://www.gitguardian.com/state-of-secrets-sprawl-report-2025. Accessed 2026-10-09.

[29] GitGuardian. ggshield secret scan ai-hook. ggshield documentation, 2026. URL https://docs.gitguardian.com/ggshield-docs/reference/secret/scan/ai-hook. Accessed 2026-10-09.

[30] GitGuardian. The state of secrets sprawl 2026, 2026. URL https://www.gitguardian.com/state-of-secrets-sprawl-report-2026. 5th edition (covers 2025 data). Accessed 2026-10-09.

[31] GitHub. actions/runner: `ValueEncoders.cs` (secret-masker value encoders). Source code, 2019. URL https://github.com/actions/runner/blob/main/src/Sdk/DTLogging/Logging/ValueEncoders.cs. Registered in `src/Runner.Common/HostContext.cs`; present since the initial commit, 2019-10-10. Accessed 2026-10-09.

[32] GitHub. Secure use reference (GitHub Actions). GitHub Docs, 2026. URL https://docs.github.com/en/actions/reference/security/secure-use. Accessed 2026-10-09.

[33] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985.

[34] Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhi Chen, and Xiuzhen Cheng. A measurement study of Model Context Protocol ecosystem, 2025. URL https://arxiv.org/abs/2509.25292.

[35] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers, 2025. URL https://arxiv.org/abs/2506.13538.

[36] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model Context Protocol (MCP): Landscape, security threats, and future research directions, 2025. URL https://arxiv.org/abs/2503.23278.

[37] Xinyi Hou, Yanjie Zhao, and Haoyu Wang. Unsafe by flow: Uncovering bidirectional data-flow risks in MCP ecosystem, 2026. URL https://arxiv.org/abs/2605.07836.

[38] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. Your code secret belongs to me: Neural code completion tools can memorize hard-coded credentials. *Proceedings of the ACM on Software Engineering*, 1(FSE):2515–2537, 2024. doi: 10.1145/3660818.

[39] Infisical. Infisical MCP server. GitHub repository Infisical/infisical-mcp-server, 2025. URL https://github.com/Infisical/infisical-mcp-server. See INFISICAL_MASK_SECRET_VALUES. Accessed 2026-10-09.

[40] Infisical. Agent proxy. Infisical documentation, 2026. URL https://infisical.com/docs/documentation/platform/agent-proxy/overview. Accessed 2026-10-09.

[41] Infisical. CLI reference: infisical run, 2026. URL https://infisical.com/docs/cli/commands/run. Accessed 2026-10-09.

[42] Invariant Labs. MCP security notification: Tool poisoning attacks, April 2025. URL https://invariantlabs.ai/blog/mcp-security-notification. Published 2025-04-01. Accessed 2026-10-09.

[43] Patrick Kenney, Hadi Ahmadi, Denis Lusson, Donald Nguyen, and Gurbinder Gill. API secrets should never become tokens in the LLM's vocabulary: A threat analysis of API credential handling in LLM agent systems and an empirical evaluation of a vault-mediated execution boundary, 2026. URL https://arxiv.org/abs/2609.33371.

[44] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. Characterizing the security of GitHub CI workflows. In *31st USENIX Security Symposium (USENIX Security 22)*, 2022. URL https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev.

[45] LangWatch. Secrets redaction in coding agent traces, 2026. URL https://langwatch.ai/docs/coding-agents/secrets-redaction. Accessed 2026-10-09.

[46] Linux man-pages project. network_namespaces(7) — overview of linux network namespaces, 2026. URL https://man7.org/linux/man-pages/man7/network_namespaces.7.html. Accessed 2026-10-09.

[47] Linux man-pages project. user_namespaces(7) — overview of linux user namespaces, 2026. URL https://man7.org/linux/man-pages/man7/user_namespaces.7.html. Accessed 2026-10-09.

[48] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024. URL https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei.

[49] Jaroslav Lobačevski. Keeping your GitHub Actions and workflows secure part 1: Preventing pwn requests. GitHub Security Lab, August 2021. URL https://securitylab.github.com/resources/github-actions-preventing-pwn-requests/. Accessed 2026-10-09.

[50] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public GitHub repositories. In *Network and Distributed System Security Symposium (NDSS)*, 2019. doi: 10.14722/ndss.2019.23418.

[51] Meta AI. Agents rule of two: A practical approach to AI agent security. Meta AI blog, October 2025. URL https://ai.meta.com/blog/practical-ai-agent-security/. Published 2025-10-31. Accessed 2026-10-09.

[52] Model Context Protocol. Model Context Protocol specification (version 2026-07-28), 2026. URL https://modelcontextprotocol.io/specification/2026-07-28. Accessed 2026-10-09.

[53] Dor Munis. Claude Code automatically loads .env secrets, without telling you. Knostic blog, December 2025. URL https://www.knostic.ai/blog/claude-loads-secrets-without-permission. Published 2025-12-08. Accessed 2026-10-09.

[54] Ishan Naik. agent-sweep: Find and redact secrets in AI coding agent histories. GitHub repository Ishannaik/agent-sweep, 2026. URL https://github.com/Ishannaik/agent-sweep. Accessed 2026-10-09.

[55] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections, 2025. URL https://arxiv.org/abs/2510.09023.

[56] nbyk115. [feature request] allow a PostToolUse hook to modify (redact) the output string. GitHub issue anthropics/claude-code#64326, 2026. URL https://github.com/anthropics/claude-code/issues/64326. Title translated from Japanese. Opened 2026-05-31; closed. Accessed 2026-10-09.

[57] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, pages 2353–2367, 2022. doi: 10.1145/3548606.3560596.

[58] OneCLI. OneCLI: open-source sandboxed agent harness with credential-injecting gateway. GitHub repository onecli/onecli, 2026. URL https://github.com/onecli/onecli. Accessed 2026-10-09.

[59] Alexander Panfilov, David Schmotz, Ilia Shumailov, Luca Beurer-Kellner, Joachim Schaeffer, Ameya Prabhu, Jonas Geiping, and Maksym Andriushchenko. Stealing reasoning traces from proprietary LLM APIs, 2026. URL https://arxiv.org/abs/2608.09867.

[60] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. URL https://arxiv.org/abs/2211.09527.

[61] PrismorSec. Prismor: self-hosted runtime control plane for AI agents (Sweep and Cloak). GitHub repository (formerly immunity-agent), 2026. URL https://github.com/PrismorSec/prismor. Accessed 2026-10-09.

[62] PrismorSec. Human approvals, 2026. URL https://www.prismor.dev/docs/approvals. Current unversioned documentation. Accessed 2026-10-21.

[63] PrismorSec. Decision contract, 2026. URL https://www.prismor.dev/docs/decision-contract. Current unversioned documentation. Accessed 2026-10-21.

[64] r3352. redact-mcp: MCP server that auto-obfuscates sensitive data. GitHub repository, 2026. URL https://github.com/r3352/redact-mcp. Accessed 2026-10-09.

[65] Brandon Radosevich and John Halloran. MCP safety audit: LLMs with the Model Context Protocol allow major security exploits, 2025. URL https://arxiv.org/abs/2504.03767.

[66] Mohammadreza Rashidi. The balkanization of execution-security research for AI coding agents: Isolation, access control, and time-of-check-to-time-of-use vulnerabilities, 2026. URL https://arxiv.org/abs/2607.05743.

[67] sainitish1609. mcp-guard: local security firewall and secret sanitizer for MCP. GitHub repository, 2026. URL https://github.com/sainitish1609/mcp-guard. Accessed 2026-10-09.

[68] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999. doi: 10.1145/317087.317089.

[69] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. URL https://arxiv.org/abs/2504.11703v3. Version 3, revised 2026-05-14; v1/v2 titled "Progent: Programmable Privilege Control for LLM Agents".

[70] The Linux Kernel documentation. Seccomp BPF (SECure COMPuting with filters), 2026. URL https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html. Accessed 2026-10-09.

[71] TrueFoundry. AI gateway: Secrets detection guardrail, 2026. URL https://www.truefoundry.com/docs/ai-gateway/secrets-detection. Accessed 2026-10-09.

[72] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A benchmark for tool poisoning on real-world MCP servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 35811–35819, 2026. doi: 10.1609/aaai.v40i42.40895.

[73] Simon Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. Blog post, April 2023. URL https://simonwillison.net/2023/Apr/25/dual-llm-pattern/. Accessed 2026-10-09.

[74] Simon Willison. The lethal trifecta for AI agents: private data, untrusted content, and external communication. Blog post, June 2025. URL https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/. Accessed 2026-10-09.

[75] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024. URL https://arxiv.org/abs/2409.19091.

[76] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.241131.

[77] Haocheng Xia and Yongjoo Park. SlotGuard: Stop oversharing private local context in LLM agent transcripts. In *ICML 2026 Workshop (AIWILD)*, 2026. URL https://arxiv.org/abs/2607.17147.

[78] Yuchong Xie, Mingyu Luo, Zesen Liu, Zhixiang Zhang, Kaikai Zhang, Yu Liu, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. Red-teaming coding agents from a tool-invocation perspective: An empirical security assessment, 2025. URL https://arxiv.org/abs/2509.05755.

[79] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, 2019. URL https://www.usenix.org/conference/hotcloud19/presentation/young.

[80] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents.

In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.624. URL https://aclanthology.org/2024.findings-acl.624/.

[81] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. Parasites in the toolchain: A large-scale analysis of attacks on the MCP ecosystem. In *IEEE Symposium on Security and Privacy (S&P)*, 2026. doi: 10.1109/SP63933.2026.00154. URL https://arxiv.org/abs/2509.06572.

[82] Huijun Zhou, Xiaohan Zhang, Haozhe Zhang, Haoyang Zhang, Mi Zhang, and Min Yang. A first measurement study on authentication security in real-world remote MCP servers, 2026. URL https://arxiv.org/abs/2605.22333.